

Security and Data Governance

Architecture section: trust boundaries and the two data-protection control points

Lizhan Hong

Orange Wholesale (OW / OINIS)

Contents

1. Trust domains	1
2. Invariants	2
3. The two control points	2
4. Mapping to the security review questions	3
5. Implementation status	4

This section formalises the data-flow guarantees of the network digital-twin project so that documentation, code and the statements made to security review stay aligned. It defines the trust domains, the invariants that always hold, and the two control points that govern any use of an external language model. It is meant to be read together with the trust-boundary diagram.

1. Trust domains

The system is partitioned into three domains with different sensitivity, and data movement between them is one-directional and controlled.

Production domain. The production configuration repository and the NUAR telemetry platform. This domain is sensitive and is never entered by the agents. The only outbound movements are the production configuration being read once, inside the LAB, to generate the twin, and historical NUAR counters being used offline in the LAB for model training and validation. There is no live agent connection to production.

LAB domain. An isolated environment where all agent computation takes place: the generated EVE-NG twin, the machine-generated synthetic data, the detection and root-cause agents, the offline-trained predictor, and the default processing core. The default core runs entirely locally with no external calls.

Optional LLM layer. A language model is an optional enhancement placed behind an interface. It is not part of the default system. When it is enabled, everything leaving the

LAB toward it passes through the two control points defined in Section 3.

2. Invariants

These properties hold in every configuration of the system, including with the LLM layer enabled.

The agents never open a connection to the production domain; they operate on the twin, which is a copy. The agents read the live twin read-only and mutate only a sandbox copy for counterfactual tests. The twin reproduces network structure and behaviour, that is topology, routing, configuration logic and load patterns, and does not carry customer traffic; the data the agents reason over is operational metadata such as link utilisation, adjacency and routing state, and configuration diffs, with no user payload and no personal data. Customer reachability is represented by prefix identifiers, not customer identities. The default core, meaning the rule-based root-cause reasoner, the analytic performance model and the evaluator, runs fully locally and makes zero external calls, so the system is operable with no external LLM at all. Synthetic data is machine-generated by the twin's traffic model and fault injector, is deterministic and reproducible from a fixed seed, and is not derived from real production traces.

3. The two control points

Both are permanent architectural seams. They are present in the system at all times and are exercised on the path to the LLM layer. When no LLM is used, no data reaches them and they are inert.

3.1 Control point 1: pseudonymisation and minimisation

Placement. On the boundary between the LAB core and the LLM layer. Any context assembled for a language model passes through it first.

Function. Two operations. Minimisation reduces the context to a whitelist of fields required for the task, so raw configuration text is never forwarded, only the structural and metadata fields needed for reasoning. Pseudonymisation replaces identifiers, that is device names, interface identifiers, IP addresses and customer prefixes, with stable pseudonyms drawn from the project's canonical identity space, so that the model sees R1, LINK-7, PFX-3 rather than real names. The mapping is deterministic within a run and is kept in the LAB, which allows the model's output to be mapped back to real identifiers locally for display without the real identifiers ever leaving.

Guarantee. What is presented to a language model contains pseudonymised operational metadata only, never raw production configuration and never customer-identifying information.

Interface.

```
class Pseudonymiser(ABC):
```

```
def redact(self, context) -> tuple[RedactedContext, Mapping]
def restore(self, text, mapping) -> str          # local display only
```

The default implementation is the identity map, a no-op, so the offline default path is unchanged. The active implementation is used whenever the LLM layer is enabled.

3.2 Control point 2: egress filtering and audit

Placement. A mandatory gate around every outbound call to any endpoint outside the LAB. All language-model calls, both the reasoning agent and the evaluation judge, route through it; there is no other path out.

Function. The gate classifies the destination as local, internally hosted, or external, and applies policy per class. It verifies that the payload has passed pseudonymisation, rejects any payload that still matches sensitive patterns such as raw configuration markers, non-pseudonymised addresses or credential-like tokens, and enforces size and rate limits. An external destination requires an explicit authorisation flag; a public or unknown destination is blocked. Every attempt, allowed or blocked, is written to an append-only audit log recording the timestamp, the destination class, a content hash, the size, whether the content was pseudonymised, the decision and the reason.

Guarantee. No content leaves the LAB except through this point; nothing reaches a public interface; and every outbound attempt leaves an auditable record. If the gate blocks a call, the system falls back to its local rule-based path and logs the event, so a blocked egress degrades capability rather than leaking data or failing open.

Interface.

```
class EgressGuard(ABC):
    def check(self, payload, destination) -> Decision # allow | block + reasons
# every external call is wrapped: pseudonymise -> guard.check -> (if allow) call
```

The recommended deployment sends allowed traffic to an internally hosted or private-VPC model with no retention and no training on the data. The external-endpoint path exists only if security policy authorises it, and the public-forum path is blocked by construction.

4. Mapping to the security review questions

The first review raised four questions; the design answers each by construction.

On the origin of input data, both agents operate in the LAB scope: the twin is a LAB copy generated from production configuration, and NUAR history is used offline in the LAB. On anonymisation, the twin is itself a structural abstraction carrying no customer traffic, and control point 1 pseudonymises identifiers and drops raw configuration before any model sees the context. On the synthetic NUAR data, it is machine-generated by the twin's traffic model and fault injector, reproducible from a seed, and not derived from real traces. On external language-model APIs and data export, the system runs offline by default with no external call, the LLM is an optional layer, and when used, control

point 2 restricts destinations to internally hosted or authorised endpoints, blocks public interfaces, and audits every outbound message.

5. Implementation status

The distinction below is deliberate, so that what is claimed matches what is built.

Element	Status
Agents confined to the LAB twin, read-only live, sandbox for tests	implemented
Default core fully local, zero external calls, no key required	implemented
Synthetic data machine-generated, deterministic, reproducible	implemented
Reasoning over operational metadata only, prefixes not customer identities	implemented
Control point 1: pseudonymisation and minimisation seam	implemented as a seam, active when the LLM layer is enabled
Control point 2: egress filtering and audit gate	implemented as a seam, active when the LLM layer is enabled
Internally hosted or private-VPC model deployment	deployment choice, subject to security policy

The two control points are always present in the code as the single path to any external model, and are exercised whenever that optional layer is turned on. Their default behaviour is inert because the default system makes no external call. This keeps three things consistent: the architecture as documented here, the behaviour as implemented in the code, and the guarantees as stated to security review.